



创新创业实践

libsecp256k1 中 ECDSA 低 s 规范化、
签名延展性与消息哈希检查分析

实验成员：董胥宏

学 号：202300460176

实验平台：Ubuntu 22.04.5 LTS

完成时间：2026 年 7 月 21 日

摘要

本实验以 Bitcoin Core 使用的高性能椭圆曲线密码库 libsecp256k1 为研究对象, 围绕 ECDSA 签名验证中的两个安全问题展开: 第一, 验证者若不自行对原始消息进行密码学哈希, 而是直接接受外部提供的 32 字节哈希值, 攻击者可在不知道私钥的情况下构造能够通过验证的“哈希值-签名”组合; 第二, 标准 ECDSA 具有签名延展性, 对有效签名 (r, s) , $(r, n - s)$ 在基础数学验证关系下仍可能有效。

实验首先下载并编译 bitcoin-core/secp256k1 官方源码, 运行其 207 项官方测试, 全部通过。随后分析 secp256k1_ecdsa_verify() 与 secp256k1_ecdsa_signature_normalize() 的实现, 并编写 C 程序构造高 s 签名。实验结果表明: 原始低 s 签名验证成功; 构造得到的高 s 签名被公开验证接口拒绝; 经规范化转换后, 签名恢复为原始低 s 形式并重新通过验证。实验验证了 libsecp256k1 对签名延展性的防护机制, 也说明了“验证者自行计算消息哈希”和“只接受低 s 签名”的必要性。

关键词: ECDSA; secp256k1; 签名延展性; 低 s 规范化; 比特币; 椭圆曲线密码学

目录

1	实验背景	1
2	实验目的	1
3	实验环境	2
4	ECDSA 数学基础	2
4.1	密钥生成	2
4.2	签名过程	3
4.3	验证过程	3
5	未检查原始消息时的签名伪造	4
5.1	问题描述	4
5.2	构造过程	4
6	ECDSA 签名延展性与低 s 规则	5
6.1	高 s 等价签名	5
6.2	低 s 规范	6

7	libsecp256k1 源码获取、构建与测试	6
7.1	获取官方源码	6
7.2	配置与编译	7
7.3	运行官方测试	8
8	关键源码分析	9
8.1	验证者必须自行计算消息哈希	9
8.2	低 s 规范化函数	10
8.3	验证函数中的低 s 检查	11
9	实验程序设计	13
9.1	总体流程	13
9.2	紧凑签名格式	13
9.3	大端整数减法	13
9.4	编译命令	14
10	实验结果与分析	14
10.1	运行结果	14
10.2	数值关系	15
10.3	结果解释	16
11	安全修复与性能设计分析	16
11.1	安全意义	16
11.1.1	应用层：绑定原始消息	16
11.1.2	密码库层：低 s 规范化	17
11.2	性能优化设计概述	17
12	实验中遇到的问题与解决方法	18
13	实验结论	18

1 实验背景

比特币使用椭圆曲线密码学实现公钥生成和数字签名。`libsecp256k1` 是 Bitcoin Core 使用的专用 `secp256k1` 椭圆曲线密码库，提供有限域运算、椭圆曲线点运算、密钥生成、ECDSA 签名与验证等功能。

本实验对应的课程任务要求分析比特币密码算法使用中的安全修复或性能改进，说明修改原因及其数学原理。结合课件中“ECDSA – Forge signature when the signed message is not checked”的内容，本实验重点研究以下两个问题：

1. 为什么验证者不能直接信任外部传入的消息哈希；
2. 为什么 `libsecp256k1` 只接受低 s 形式的 ECDSA 签名；
3. 如何将高 s 签名规范化为低 s ；
4. 上述安全设计背后的椭圆曲线和模运算原理。

2 实验目的

1. 掌握 `secp256k1` 曲线下 ECDSA 的密钥生成、签名与验证过程；
2. 理解未检查原始消息时的 ECDSA 存在性伪造原理；
3. 理解 ECDSA 签名延展性及 (r, s) 与 $(r, n - s)$ 的关系；
4. 分析 `libsecp256k1` 中低 s 检查与规范化函数的源码；
5. 编写程序构造高 s 签名，验证库的拒绝和规范化行为；
6. 形成可复现的实验材料并上传至小组 GitHub 总仓库。

3 实验环境

表 1 实验环境与工具

项目	配置
操作系统	Ubuntu 22.04.5 LTS
Git	2.34.1
GCC	11.4.0
CMake	3.22.1
GNU Make	4.3
实验库	bitcoin-core/secp256k1
源码提交	8c3e6e6d992456d3b9228305ae84a6703273cf70
编程语言	C11
构建类型	RelWithDebInfo

```
(base) xingye@ubuntu:~/ecdsa-homework$ echo "===== EXPERIMENT ENVIRONMENT ====="
echo "Current directory: $(pwd)"
cat /etc/os-release | grep PRETTY_NAME
git --version
gcc --version | head -n 1
cmake --version | head -n 1
make --version | head -n 1
===== EXPERIMENT ENVIRONMENT =====
Current directory: /home/xingye/ecdsa-homework
PRETTY_NAME="Ubuntu 22.04.5 LTS"
git version 2.34.1
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
cmake version 3.22.1
GNU Make 4.3
```

图 1 Ubuntu 实验环境及开发工具版本

图中显示 Ubuntu、Git、GCC、CMake 和 GNU Make 均已正确安装，实验工作目录为 `/home/xingye/ecdsa-homework`。

4 ECDSA 数学基础

4.1 密钥生成

设 G 为 secp256k1 椭圆曲线上的基点， n 为 G 的阶，私钥为整数 d ，满足

$$1 \leq d \leq n - 1.$$

公钥为

$$P = dG.$$

这里的乘法表示椭圆曲线上的标量乘法，即将点 G 重复相加 d 次。

4.2 签名过程

对消息 m 计算哈希：

$$e = H(m).$$

签名者随机选择一次性随机数

$$k \in \{1, 2, \dots, n-1\},$$

计算

$$R = kG = (x_R, y_R),$$

并令

$$r = x_R \bmod n.$$

随后计算

$$s = k^{-1}(e + rd) \bmod n.$$

最终签名为

$$\sigma = (r, s).$$

随机数 k 必须保密且不可重复。若两个签名重复使用同一 k ，可能通过联立方程恢复私钥 d 。

4.3 验证过程

验证者根据消息重新计算

$$e = H(m),$$

并计算

$$w = s^{-1} \bmod n,$$

$$u_1 = ew \bmod n, \quad u_2 = rw \bmod n.$$

然后计算椭圆曲线点

$$R' = u_1G + u_2P = (x', y').$$

若

$$r \equiv x' \pmod{n},$$

则签名通过验证。

正确性可由下式说明：

$$\begin{aligned} R' &= es^{-1}G + rs^{-1}P \\ &= s^{-1}(eG + rdG) \\ &= s^{-1}(e + rd)G \\ &= kG = R. \end{aligned}$$

5 未检查原始消息时的签名伪造

5.1 问题描述

安全的签名接口应当接收原始消息 m ，并由验证者在受控环境中计算 $e = H(m)$ 。若接口允许攻击者直接提交任意的 32 字节哈希值 e ，而不检查该值是否来自合法消息，就可能产生存在性伪造。

这里的“伪造”不是恢复私钥，也不表示攻击者能够为任意指定消息签名，而是攻击者能够构造某个哈希值 e' 及其对应的有效签名 (r', s') 。

5.2 构造过程

攻击者选择非零标量

$$u, v \in \mathbb{F}_n^*,$$

并利用公开的公钥 P 计算

$$R' = uG + vP = (x', y').$$

令

$$r' = x' \bmod n.$$

为了使验证方计算出的点恰好为 R' ，要求

$$s'^{-1}(e'G + r'P) = uG + vP.$$

比较 G 与 P 的系数, 可得

$$s'^{-1}e' = u, \quad s'^{-1}r' = v.$$

因此

$$s' = r'v^{-1} \bmod n,$$

并有

$$e' = us' = r'uv^{-1} \bmod n.$$

于是 (r', s') 能够通过针对哈希值 e' 的 ECDSA 验证。这一结果说明: 验证者不能把外部提供的任意字节串直接当作已经验证来源的消息哈希, 而应当自行完成消息编码、上下文绑定和密码学哈希。

6 ECDSA 签名延展性与低 s 规则

6.1 高 s 等价签名

secp256k1 基点的阶为

$$n = \text{FF}$$

$$\text{BAAEDCE6AF48A03BBFD25E8CD0364141}.$$

对于一个有效签名 (r, s) , 构造

$$s' = n - s.$$

由于

$$s' \equiv -s \pmod{n},$$

所以

$$(s')^{-1} \equiv -s^{-1} \pmod{n}.$$

使用 s' 验证时得到的点为

$$\begin{aligned} R'' &= (s')^{-1}(eG + rP) \\ &= -s^{-1}(eG + rP) \\ &= -R. \end{aligned}$$

椭圆曲线上点 $R = (x, y)$ 的相反点是

$$-R = (x, -y),$$

二者横坐标相同，因此仍可得到同一个 r 。这说明基础 ECDSA 数学关系不能唯一确定 s 。

6.2 低 s 规范

为了使每个签名只保留一个规范形式，通常规定

$$1 \leq s \leq \frac{n}{2}.$$

若

$$s > \frac{n}{2},$$

则将其替换为

$$s_{\text{low}} = n - s.$$

这样 (r, s) 和 $(r, n - s)$ 中只保留低 s 的一个，可以减少签名层面的可变表示，避免同一授权行为出现两个不同签名字节串。

在比特币交易中，签名字节属于交易序列化数据的一部分。若允许任意一方把 s 替换为 $n - s$ ，就可能改变交易的序列化形式及相应标识符，影响依赖交易标识符的上层逻辑。因此低 s 规则属于重要的规范化和抗延展性设计。

7 libsecp256k1 源码获取、构建与测试

7.1 获取官方源码

使用 Git 克隆官方项目：

```
1 cd ~/ecdsa-homework
```

```

2 git clone https://github.com/bitcoin-core/secp256k1.git
3 cd secp256k1
4 git remote -v
5 git log -1

```

```

===== SECP256K1 REPOSITORY =====
Current directory: /home/xingye/ecdsa-homework/secp256k1

----- Remote repository -----
origin https://github.com/bitcoin-core/secp256k1.git (fetch)
origin https://github.com/bitcoin-core/secp256k1.git (push)

----- Current branch -----
master

----- Latest commit -----
Commit: 8c3e6e6d992456d3b9228305ae84a6703273cf70
Date: 2026-07-18 00:46:21 +0200
Subject: Merge bitcoin-core/secp256k1#1889: field: serialize elements by word

----- Project files -----
autogen.sh      CMakeLists.txt      COPYING            Makefile.am      tools
autotools-aux   CMakePresets.json  doc                README.md
CHANGELOG.md    configure.ac        examples           sage
ci              contrib             include            SECURITY.md
cmake           CONTRIBUTING.md     libsecp256k1.pc.in src

```

图 2 secp256k1 官方源码仓库及版本信息

本次实验使用的提交为:

8c3e6e6d992456d3b9228305ae84a6703273cf70.

7.2 配置与编译

使用 CMake 启用官方测试并进行带调试信息的优化构建:

```

1 cmake -S . -B build \
2   -DCMAKE_BUILD_TYPE=RelWithDebInfo \
3   -DSECP256K1_BUILD_TESTS=ON
4
5 cmake --build build -j"${nproc}"

```

```
===== SECP256K1 BUILD RESULT =====
Source commit: 8c3e6e6
Build directory: /home/xingye/ecdsa-homework/secp256k1/build

----- Build configuration -----
CMAKE_BUILD_TYPE:STRING=RelWithDebInfo
SECP256K1_BUILD_TESTS:BOOL=ON

----- Generated library files -----
build/lib/libsecp256k1.so.6.0.2

Build completed successfully.
```

图 3 secp256k1 项目的 CMake 配置与编译结果

构建成功后生成动态链接库 `build/lib/libsecp256k1.so.6.0.2`。

7.3 运行官方测试

执行：

```
1 ctest --test-dir build --output-on-failure
```

```
===== SECP256K1 OFFICIAL TEST RESULT =====
Source commit: 8c3e6e6

      Start 205: secp256k1.tests.secp256k1_byteorder_tests
205/207 Test #205: secp256k1.tests.secp256k1_byteorder_tests .....
.....   Passed    0.00 sec
      Start 206: secp256k1.tests.cmov_tests
206/207 Test #206: secp256k1.tests.cmov_tests .....
.....   Passed    0.00 sec
      Start 207: secp256k1.exhaustive_tests
207/207 Test #207: secp256k1.exhaustive_tests .....
.....   Passed    9.21 sec

100% tests passed, 0 tests failed out of 207

Label Time Summary:
secp256k1_exhaustive      =  9.21 sec*proc (1 test)
secp256k1_noverify_tests  = 22.78 sec*proc (103 tests)
secp256k1_tests           = 51.41 sec*proc (103 tests)

Total Test time (real) = 83.50 sec
```

图 4 secp256k1 官方测试套件运行结果

测试结果为

207/207

项测试全部通过，失败数为 0，总测试时间约 83.50 秒。这表明实验所使用的库已正确构建，官方功能测试未发现异常。

8 关键源码分析

8.1 验证者必须自行计算消息哈希

在 `include/secp256k1.h` 中，ECDSA 验证接口的注释明确说明：验证者必须自行对消息使用密码学哈希函数，不能直接接受来源不可信的 `msghash32`。否则，攻击者可能在不知道私钥的情况下构造有效的“哈希值-签名”组合。

同一段注释还说明，为避免接受可延展签名，公开验证接口仅接受低 s 形式。

```

===== ECDSA VERIFICATION SECURITY WARNING =====
File: include/secp256k1.h

    600 *          0: incorrect or unparseable signature
    601 *  Args:   ctx:      pointer to a context object
    602 *  In:      sig:      the signature being verified.
    603 *          msghash32: the 32-byte message hash being verified.
    604 *          The verifier must make sure to apply a cryptographic
    605 *          hash function to the message by itself and not accept an
    606 *          msghash32 value directly. Otherwise, it would be
    607 *          easy to create a "valid" signature without knowledge of
    608 *          the secret key. See also
    609 *          https://bitcoin.stackexchange.com/a/81116/35586
    610 *          for more background on this topic.
    611 *          pubkey:     pointer to an initialized public key to verify with.
    612 *
    613 * To avoid accepting malleable signatures, only ECDSA signatures in lower-S
    614 * form are accepted.
    615 *
    616 * If you need to accept ECDSA signatures from sources that do not obey this
    617 * rule, apply secp256k1_ecdsa_signature_normalize to the signature prior to
    618 * verification, but be aware that doing so results in malleable signatures.

```

图 5 secp256k1 对消息哈希检查 and 低 s 签名的安全说明

该安全警告与课件中的伪造推导相对应。库只能验证输入的数学关系是否成立，但无法替代应用层确认：“这个哈希究竟对应哪条原始消息、采用何种编码、属于哪个业务上下文”。

8.2 低 s 规范化函数

secp256k1_ecdsa_signature_normalize() 的核心逻辑为：

```

1 secp256k1_ecdsa_signature_load(ctx, &r, &s, sign);
2 ret = secp256k1_scalar_is_high(&s);
3
4 if (sigout != NULL) {
5     if (ret) {
6         secp256k1_scalar_negate(&s, &s);

```

```
7     }  
8     secp256k1_ecdsa_signature_save(sigout, &r, &s);  
9 }  
10 return ret;
```

其中：

- `secp256k1_scalar_is_high(&s)` 判断 s 是否大于 $n/2$;
- 若为高 s , `secp256k1_scalar_negate(&s,&s)` 执行模 n 取负, 即计算 $n - s$;
- 函数返回值表示输入签名是否发生了规范化改变。

8.3 验证函数中的低 s 检查

`secp256k1_ecdsa_verify()` 的关键返回语句为：

```
1 return (!secp256k1_scalar_is_high(&s) &&  
2        secp256k1_pubkey_load(ctx, &q, pubkey) &&  
3        secp256k1_ecdsa_sig_verify(&r, &s, &q, &m));
```

三个条件分别表示：

1. s 必须不是高 s ;
2. 公钥必须能够成功解析;
3. ECDSA 数学验证必须通过。

由于 C 语言中的逻辑与运算符具有短路特性, 当第一项发现 s 为高 s 时, 公开接口直接返回失败。因此, 即使 $(r, n - s)$ 满足基础 ECDSA 数学关系, `libsecp256k1` 也会基于规范化策略拒绝该签名。

```
===== LOW-S NORMALIZATION AND VERIFICATION =====
File: src/secp256k1.c

456 }
457
458 int secp256k1_ecdsa_signature_normalize(const secp256k1_context* ctx, secp256k1_ecdsa_signature *sigout, const secp256k1_ecdsa_signature *signin) {
459     secp256k1_scalar r, s;
460     int ret = 0;
461
462     VERIFY_CHECK(ctx != NULL);
463     ARG_CHECK(signin != NULL);
464
465     secp256k1_ecdsa_signature_load(ctx, &r, &s, signin);
466     ret = secp256k1_scalar_is_high(&s);
467     if (sigout != NULL) {
468         if (ret) {
469             secp256k1_scalar_negate(&s, &s);
470         }
471         secp256k1_ecdsa_signature_save(sigout, &r, &s);
472     }
473
474     return ret;
475 }
476
477 int secp256k1_ecdsa_verify(const secp256k1_context* ctx, const secp256k1_ecdsa_signature *sig, const unsigned char *msghash32, const secp256k1_pubkey *pubkey) {
478     secp256k1_ge q;
479     secp256k1_scalar r, s;
480     secp256k1_scalar m;
481     VERIFY_CHECK(ctx != NULL);
482     ARG_CHECK(msghash32 != NULL);
483     ARG_CHECK(sig != NULL);
484     ARG_CHECK(pubkey != NULL);
485
486     secp256k1_scalar_set_b32(&m, msghash32, NULL);
487     secp256k1_ecdsa_signature_load(ctx, &r, &s, sig);
488     return (!secp256k1_scalar_is_high(&s) &&
489             secp256k1_pubkey_load(ctx, &q, pubkey) &&
490             secp256k1_ecdsa_sig_verify(&r, &s, &q, &m));
491 }
492
493 static SECP256K1_INLINE void buffer_append(unsigned char *buf, unsigned int *offset, const void *data, unsigned int len) {
```

图6 secp256k1 低 s 规范化与签名验证源码

9 实验程序设计

9.1 总体流程

实验程序 `ecdsa_low_s_demo.c` 的流程如下：

1. 创建 `secp256k1` 上下文；
2. 使用固定教学私钥 $d = 1$ 生成公钥；
3. 使用固定 32 字节消息哈希生成 ECDSA 签名；
4. 将签名序列化为 64 字节紧凑格式；
5. 验证原始低 s 签名；
6. 计算 $s_{\text{high}} = n - s$ 并构造高 s 签名；
7. 调用公开验证接口验证高 s 签名；
8. 调用规范化函数把高 s 转换为低 s ；
9. 再次验证并比较规范化签名与原始签名。

9.2 紧凑签名格式

实验使用 64 字节紧凑格式：

$$\text{compact}[0..31] = r,$$

$$\text{compact}[32..63] = s.$$

程序先序列化原始签名，然后只修改后 32 字节的 s ，保持 r 不变。

9.3 大端整数减法

为计算

$$s_{\text{high}} = n - s,$$

程序实现了 32 字节大端无符号整数减法。减法从最低有效字节，即数组末尾开始，逐字节处理借位。该实现不依赖外部大整数库，便于直接展示 $n - s$ 的构造过程。

9.4 编译命令

```
1 gcc -std=c11 -Wall -Wextra -O2 \  
2   ecdsa_low_s_demo.c \  
3   -Iinclude \  
4   -Lbuild/lib \  
5   -Wl,-rpath,"$PWD/build/lib" \  
6   -lsecp256k1 \  
7   -o ecdsa_low_s_demo
```

其中, `-Iinclude` 指定头文件路径, `-Lbuild/lib` 指定库文件路径, `-lsecp256k1` 链接 `secp256k1` 库, `rpath` 用于保证程序运行时能够找到本次构建的动态库。

```
===== DEMO PROGRAM BUILD RESULT =====  
Source file:  
-rw-rw-r-- 1 xingye xingye 8.3K Jul 20 18:51 ecdsa_low_s_demo.c  
  
Executable file:  
-rwxrwxr-x 1 xingye xingye 17K Jul 20 18:52 ecdsa_low_s_demo  
  
ecdsa_low_s_demo: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynam  
ically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=8905f2082f  
033dacd031e79bf00b34604c310b01, for GNU/Linux 3.2.0, not stripped  
  
Linked secp256k1 library:  
    libsecp256k1.so.6 => /home/xingye/ecdsa-homework/secp256k1/build/lib/lib  
secp256k1.so.6 (0x0000778fe7b1c000)
```

图 7 ECDSA 低 s 实验程序的编译与动态链接结果

图中可见, 可执行文件为 x86-64 ELF 程序, 并且实际链接到本次实验构建目录中的动态链接库:

build/lib/libsecp256k1.so.6

因此, 可以确认实验程序使用的是本次自行编译得到的 `libsecp256k1` 库。

10 实验结果与分析

10.1 运行结果

运行命令:

```
1 ./ecdsa_low_s_demo
```

```
(base) xingye@ubuntu:~/ecdsa-homework/secp256k1$ ./ecdsa_low_s_demo
=====
libsecp256k1 ECDSA Low-S Normalization Demonstration
=====

[Signature values]
Message hash : 7A21438FC5926116FAD30C57A810E45B9137CD286E40B27519A6F803D455BC9E
R            : AAF76A4B1DA0E11A993521C060BF92C0E56230A0E93E467FD43B4019EA56E45E
Original S   : 46BFCF0D55E64A85D77B0607821826A7C70DB923B064759A0A61FF93B6BAE1D8
High S = n-S : B94030F2AA19B57A2884F9F87DE7D956F3A123C2FEE42AA1B5705EF9197B5F69
Normalized S : 46BFCF0D55E64A85D77B0607821826A7C70DB923B064759A0A61FF93B6BAE1D8

[Verification results]
[1] Original low-S signature verification : PASS
[2] High-S signature construction       : SUCCESS
[3] High-S signature verification       : REJECTED (expected)
[4] Normalization changed the signature : YES
[5] Normalized signature verification   : PASS
[6] Normalized signature equals original : YES

[Conclusion]
The experiment completed successfully.
libsecp256k1 rejected the malleable high-S signature,
and accepted it again after low-S normalization.
```

图 8 高 s 签名拒绝及低 s 规范化实验结果

程序输出的关键结果如表所示。

表 2 低 s 规范化实验结果

检查项目	结果
原始低 s 签名验证	PASS
高 s 签名构造	SUCCESS
高 s 签名公开验证	REJECTED (符合预期)
规范化是否改变签名	YES
规范化后签名验证	PASS
规范化签名是否等于原始签名	YES

10.2 数值关系

实验中得到：

$$s_{\text{original}} = 46BFCF0D55E64A85D77B0607821826A7C70DB923B064759A0A61FF93B6BAE1D8,$$

$$s_{\text{high}} = \text{B94030F2AA19B57A2884F9F87DE7D956} \\ \text{F3A123C2FEE42AA1B5705EF9197B5F69.}$$

两者满足

$$s_{\text{original}} + s_{\text{high}} = n.$$

规范化后的 s 与原始 s 完全一致，说明

$$n - (n - s) = s.$$

10.3 结果解释

实验结果符合理论预期：

1. 官方签名函数生成的是低 s 签名，因此原始验证通过；
2. 程序成功构造 $(r, n - s)$ ；
3. 公开验证接口先执行高 s 检查，因此拒绝该签名；
4. 规范化函数检测到高 s ，执行模 n 取负；
5. 规范化结果恢复为原始低 s 签名并重新通过验证。

这说明 libsecp256k1 并非只实现基础 ECDSA 方程，还在公开 API 层增加了规范化规则，以限制签名的可变表示。

11 安全修复与性能设计分析

11.1 安全意义

本实验涉及的安全设计可归纳为两个层次。

11.1.1 应用层：绑定原始消息

验证者必须明确：

- 原始消息的格式；

- 消息使用的哈希算法；
- 消息所属协议和业务上下文；
- 消息是否存在前缀、域分离标识或长度编码。

若只验证攻击者提供的任意 `msghash32`，密码库只能证明“该签名和这个数值满足数学关系”，不能证明该数值对应某条合法业务消息。

11.1.2 密码库层：低 s 规范化

低 s 规则消除了 ECDSA 中最直接的

$$s \longleftrightarrow n - s$$

双重表示。其作用并非提高离散对数问题的计算难度，而是约束签名编码，使签名表示更接近唯一，降低签名延展性对交易和上层协议的影响。

11.2 性能优化设计概述

虽然本实验的核心验证对象是安全规范化机制，`libsecp256k1` 同时包含多项面向比特币场景的性能设计：

1. **专用 256 位有限域和标量表示**：针对 `secp256k1` 的固定素数域和固定群阶设计运算，避免通用任意精度整数库带来的额外开销。
2. **投影坐标**：椭圆曲线点加与点倍运算可使用 Jacobian 等投影表示，把频繁的有限域求逆转换为更多乘法和平方，因为求逆通常明显慢于乘法。
3. **窗口法与预计算**：标量乘法可把标量分解为窗口表示，配合基点预计算表减少椭圆曲线点运算次数。
4. **多标量乘法**：ECDSA 验证需要计算 $u_1G + u_2P$ ，可通过联合计算减少独立标量乘法的开销。
5. **常数时间秘密操作**：私钥和签名随机数相关操作避免依赖秘密值的明显分支和内存访问模式，在提高实现安全性的同时保持专用实现的高效率。

本实验未单独进行性能基准测试，因此上述内容属于对库设计思想的分析，不将其表述为本实验测得的具体加速倍数。

12 实验中遇到的问题与解决方法

1. 源码与报告位于不同系统。

实验代码在 Ubuntu 虚拟机中运行，截图与 LaTeX 报告在 Windows 主机中整理。解决方法是仅将自编写源码、日志和截图复制到主机，不复制体积较大的官方源码和构建目录。

2. 如何证明程序使用了本次构建的库。

仅有可执行结果不能完全证明动态库来源，因此使用 `ldd ecdsa_low_s_demo` 检查动态链接路径，确认其指向实验目录中的 `build/lib/libsecp256k1.so.6`。

3. 高 s 签名被拒绝是否表示构造失败。

不是。高 s 签名被公开接口拒绝正是预期安全行为。随后的规范化成功和重新验证通过进一步证明构造过程正确。

13 实验结论

本实验完成了 libsecp256k1 的源码下载、构建、官方测试、关键接口分析和自编程验证。207 项官方测试全部通过，表明实验环境和库构建正确。

通过数学推导可知，ECDSA 中 (r, s) 与 $(r, n - s)$ 可能共享相同的 r ，从而形成签名延展性。实验程序成功从原始低 s 签名构造高 s 签名。libsecp256k1 的公开验证函数拒绝高 s 形式，规范化函数则将其恢复为原始低 s 形式，随后签名重新通过验证。

此外，头文件中的安全警告和课件推导共同说明：数字签名验证不仅要检查椭圆曲线数学关系，还必须由验证者自行确定消息编码、上下文并计算哈希。否则，攻击者可能构造出没有实际业务含义但能够通过数学验证的“哈希值-签名”组合。

综上，低 s 规范化解决的是签名表示不唯一问题，消息哈希检查解决的是签名与真实消息之间的绑定问题。两者分别位于密码库层和应用协议层，共同构成安全使用 ECDSA 的必要条件。